

HDOC DAY-18 ASSIGNMENT

Menu-Driven Digit-Based Operations Using Switch-Case

1. Problem Understanding

The task is to write a menu-driven C program using switch-case to perform four digit-based operations on a positive integer. A separate user-defined function with void return type is used for each menu option. Each function reads the required input, performs the operation, and displays the result.

- Check whether the number is a Strong number.
- Check whether the number is an Armstrong number.
- Check whether the number is a Harshad number.
- Check whether the number is a Duck number.

2. Standard Test Cases

Test Case	Choice	Input	Expected Output	Status
TC1	1	145	Strong number	Pass
TC2	1	123	Not a Strong number	Pass
TC3	2	153	Armstrong number	Pass
TC4	2	123	Not an Armstrong number	Pass
TC5	3	18	Harshad number	Pass
TC6	3	19	Not a Harshad number	Pass
TC7	4	1023	Duck number	Pass
TC8	4	1234	Not a Duck number	Pass

3. Algorithm

A. Common Menu Logic

1. Start.
2. Display the menu options.
3. Read the user's choice.
4. Use switch-case to select the required operation.
5. Call the corresponding void function.
6. The selected function reads the positive integer, performs the operation, and displays the result.
7. If the choice is invalid, display "Invalid choice".
8. Stop.

B. Check Whether the Number is a Strong Number

1. Start.
2. Read a positive integer n.

3. Store the original number in original.
4. Set sum = 0.
5. Extract the last digit using $\text{digit} = n \% 10$.
6. Set fact = 1.
7. Find the factorial of digit using a for loop.
8. Add fact to sum.
9. Remove the last digit using $n = n / 10$.
10. Repeat Steps 5–9 until n becomes 0.
11. Compare sum with original.
12. If sum equals original, display "Strong number"; otherwise display "Not a Strong number".
13. Stop.

C. Check Whether the Number is an Armstrong Number

1. Start.
2. Read a positive integer n.
3. Store the original number in original and copy n into temp.
4. Set count = 0.
5. Divide temp by 10 repeatedly and increase count by 1 each time until temp becomes 0.
6. Copy n into temp again and set sum = 0.
7. Extract the last digit using $\text{digit} = \text{temp} \% 10$.
8. Set power = 1.
9. Multiply power by digit, count times, using a for loop.
10. Add power to sum.
11. Remove the last digit using $\text{temp} = \text{temp} / 10$.
12. Repeat Steps 7–11 until temp becomes 0.
13. Compare sum with original.
14. If sum equals original, display "Armstrong number"; otherwise display "Not an Armstrong number".
15. Stop.

D. Check Whether the Number is a Harshad Number

1. Start.
2. Read a positive integer n.
3. Store the original number in original.
4. Set sum = 0.
5. Extract the last digit using $\text{digit} = n \% 10$.
6. Add digit to sum.
7. Remove the last digit using $n = n / 10$.
8. Repeat Steps 5–7 until n becomes 0.
9. Check whether original is exactly divisible by sum using $\text{original} \% \text{sum} == 0$.
10. If the remainder is 0, display "Harshad number"; otherwise display "Not a Harshad number".
11. Stop.

E. Check Whether the Number is a Duck Number

1. Start.
2. Read a positive integer n.
3. Set flag = 0.
4. Extract the last digit using $\text{digit} = n \% 10$.
5. If digit is 0, set flag = 1.

6. Remove the last digit using $n = n / 10$.
7. Repeat Steps 4–6 until n becomes 0.
8. If flag is 1, display "Duck number"; otherwise display "Not a Duck number".
9. Stop.

4. C Program

```
#include <stdio.h>
void checkStrong()
{
    int n, original, digit, i, fact, sum = 0;
    printf("Enter a positive integer: ");
    scanf("%d", &n);
    original = n;
    while (n > 0)
    {
        digit = n % 10;
        fact = 1;
        for (i = 1; i <= digit; i++)
        {
            fact = fact * i;
        }
        sum = sum + fact;
        n = n / 10;
    }
    if (sum == original)
        printf("Strong number\n");
    else
        printf("Not a Strong number\n");
}

void checkArmstrong()
{
    int n, original, temp, digit, count = 0, i;
    int sum = 0, power;
    printf("Enter a positive integer: ");
    scanf("%d", &n);
    original = n;
    temp = n;
    while (temp > 0)
    {
        count++;
        temp = temp / 10;
    }
    temp = n;
    while (temp > 0)
    {
        digit = temp % 10;
        power = 1;
        for (i = 1; i <= count; i++)
        {
            power = power * digit;
        }
    }
}
```

```

        sum = sum + power;
        temp = temp / 10;
    }
    if (sum == original)
        printf("Armstrong number\n");
    else
        printf("Not an Armstrong number\n");
}

void checkHarshad()
{
    int n, original, digit, sum = 0;
    printf("Enter a positive integer: ");
    scanf("%d", &n);
    original = n;
    while (n > 0)
    {
        digit = n % 10;
        sum = sum + digit;
        n = n / 10;
    }
    if (original % sum == 0)
        printf("Harshad number\n");
    else
        printf("Not a Harshad number\n");
}

void checkDuck()
{
    int n, digit, flag = 0;
    printf("Enter a positive integer: ");
    scanf("%d", &n);
    while (n > 0)
    {
        digit = n % 10;
        if (digit == 0)
            flag = 1;
        n = n / 10;
    }
    if (flag == 1)
        printf("Duck number\n");
    else
        printf("Not a Duck number\n");
}

int main()
{
    int choice;
    printf("MENU\n");
    printf("1. Check Strong number\n");
    printf("2. Check Armstrong number\n");
    printf("3. Check Harshad number\n");
    printf("4. Check Duck number\n");
    printf("Enter your choice: ");
    scanf("%d", &choice);
    switch (choice)

```

```

{
    case 1:
        checkStrong();
        break;

    case 2:
        checkArmstrong();
        break;

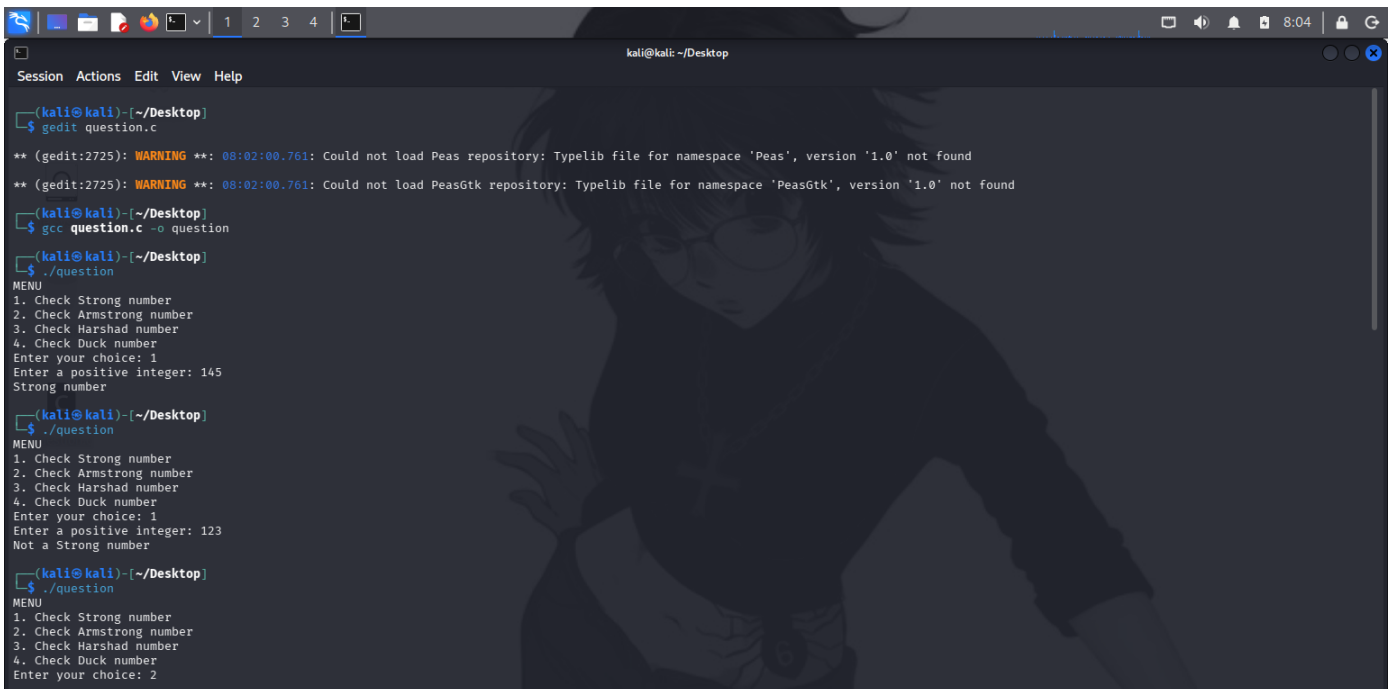
    case 3:
        checkHarshad();
        break;

    case 4:
        checkDuck();
        break;

    default:
        printf("Invalid choice\n");
}
return 0;
}

```

5. Compilation and Execution Evidence



```

kali@kali: ~/Desktop
Session Actions Edit View Help

(kali@kali)~{~/Desktop}
$ gedit question.c

** (gedit:2725): WARNING **: 08:02:00.761: Could not load Peas repository: Typelib file for namespace 'Peas', version '1.0' not found
** (gedit:2725): WARNING **: 08:02:00.761: Could not load PeasGtk repository: Typelib file for namespace 'PeasGtk', version '1.0' not found

(kali@kali)~{~/Desktop}
$ gcc question.c -o question

(kali@kali)~{~/Desktop}
$ ./question
MENU
1. Check Strong number
2. Check Armstrong number
3. Check Harshad number
4. Check Duck number
Enter your choice: 1
Enter a positive integer: 145
Strong number

(kali@kali)~{~/Desktop}
$ ./question
MENU
1. Check Strong number
2. Check Armstrong number
3. Check Harshad number
4. Check Duck number
Enter your choice: 1
Enter a positive integer: 123
Not a Strong number

(kali@kali)~{~/Desktop}
$ ./question
MENU
1. Check Strong number
2. Check Armstrong number
3. Check Harshad number
4. Check Duck number
Enter your choice: 2

```

```
kali@kali: ~/Desktop
Session Actions Edit View Help
Enter a positive integer: 153
Armstrong number

(kali@kali)-[~/Desktop]
$ ./question
MENU
1. Check Strong number
2. Check Armstrong number
3. Check Harshad number
4. Check Duck number
Enter your choice: 2
Enter a positive integer: 123
Not an Armstrong number

(kali@kali)-[~/Desktop]
$ ./question
MENU
1. Check Strong number
2. Check Armstrong number
3. Check Harshad number
4. Check Duck number
Enter your choice: 3
Enter a positive integer: 18
Harshad number

(kali@kali)-[~/Desktop]
$ ./question
MENU
1. Check Strong number
2. Check Armstrong number
3. Check Harshad number
4. Check Duck number
Enter your choice: 3
Enter a positive integer: 19
Not a Harshad number

(kali@kali)-[~/Desktop]
$ ./question
MENU
1. Check Strong number
2. Check Armstrong number
```

```
kali@kali: ~/Desktop
Session Actions Edit View Help
3. Check Harshad number
4. Check Duck number
Enter your choice: 3
Enter a positive integer: 18
Harshad number

(kali@kali)-[~/Desktop]
$ ./question
MENU
1. Check Strong number
2. Check Armstrong number
3. Check Harshad number
4. Check Duck number
Enter your choice: 3
Enter a positive integer: 19
Not a Harshad number

(kali@kali)-[~/Desktop]
$ ./question
MENU
1. Check Strong number
2. Check Armstrong number
3. Check Harshad number
4. Check Duck number
Enter your choice: 4
Enter a positive integer: 1023
Duck number

(kali@kali)-[~/Desktop]
$ ./question
MENU
1. Check Strong number
2. Check Armstrong number
3. Check Harshad number
4. Check Duck number
Enter your choice: 4
Enter a positive integer: 1234
Not a Duck number

(kali@kali)-[~/Desktop]
$
```

6. Verification of All Test Cases

Test Case	Choice	Input	Expected Result	Actual Result	Status
TC1	1	145	Strong number	Strong number	Pass
TC2	1	123	Not a Strong number	Not a Strong number	Pass
TC3	2	153	Armstrong number	Armstrong number	Pass
TC4	2	123	Not an Armstrong number	Not an Armstrong number	Pass

TC5	3	18	Harshad number	Harshad number	Pass
TC6	3	19	Not a Harshad number	Not a Harshad number	Pass
TC7	4	1023	Duck number	Duck number	Pass
TC8	4	1234	Not a Duck number	Not a Duck number	Pass
